

AI Intelligence Pipeline

Technical Architecture, Reliability & Scale Design

Purpose. Build a reproducible pipeline that collects AI ecosystem intelligence, validates source data, enriches records, resolves entities, and exports structured datasets.

1. System Overview

The implemented pipeline separates source ingestion from enrichment and export. Network-bound collectors use asynchronous I/O where appropriate; jobs/news are normalized to UTC and filtered to a strict 24-hour freshness window. Every retained signal keeps a source URL so records remain traceable.

Layer	Implemented responsibility	Output
Sources	Startup/product directories, arXiv, GitHub, job feeds, news feeds	Raw records
Ingestion	asyncio + aiohttp requests, RSS/API parsing, timeouts	Normalized source items
Quality	Schema validation, UTC date parsing, 24-hour filter, deduplication	Accepted records
Enrichment	GitHub repository/star matching and LLM extraction/fallback	Enriched records
Resolution	Canonical names, suffix normalization, aliases	Entity mappings
Delivery	JSON + CSV export and source coverage report	Submission datasets

2. Reliability & Data Quality

Freshness: publication timestamps are parsed and normalized to UTC. Jobs and news are retained only when the timestamp is within the preceding 24 hours. Records with unparseable timestamps are rejected rather than assigned a fabricated time.

Deduplication: news uses source URLs as the primary identity; jobs use normalized source URLs and a company/title fallback fingerprint. Duplicate records are removed before export.

Traceability: job and news records retain their source name and URL. The pipeline deliberately does not invent records to satisfy a target count.

Observability: *data/signal_source_report.json* records per-source fetch status, fresh counts, AI-job counts, unique counts, and duplicate removals.

3. LLM Orchestration

The extraction layer uses a multi-tier provider strategy: **Gemini** → **Groq** → **DeepSeek** → **Mock fallback**. Provider failures can trigger retries with exponential backoff and jitter before the next tier is attempted.

Large documents are bounded by the chunking layer before LLM submission. The current chunking design uses fixed maximum-size segments so oversized requests do not become a single provider payload. Structured outputs are then validated before downstream resolution/export.

4. Entity Resolution

Entity names are normalized deterministically using casing cleanup, corporate-suffix normalization, aliases, and canonical mappings. This keeps variations such as 'OpenAI', 'Open AI', and 'OpenAI Inc.' closer to one canonical entity without asking the LLM to invent an identity.

5. Scale Design: 500,000+ Records

The current repository is a modular implementation suitable for local execution. At 500k+ records, the same stages can be deployed as horizontally scaled workers without changing the logical data contract:

- Queue-based ingestion:** place source tasks on Kafka/RabbitMQ/SQS-style queues and partition work by source/domain.
- Worker scaling:** run independent crawler, enrichment, and resolution workers; autoscale using queue depth and resource utilization.
- Rate control:** maintain per-domain/provider concurrency and rate-limit state centrally so multiple workers do not overload one source.
- Idempotency:** use deterministic URL/content fingerprints so retries and worker restarts do not create duplicate records.
- Storage:** PostgreSQL for structured records, pgvector for semantic retrieval, and Neo4j when relationship-heavy entity queries justify a graph store.
- Observability:** collect source latency, error rates, freshness, queue depth, retry counts, and record acceptance/rejection metrics.

6. Failure Handling

- 429 / rate limits:** exponential backoff with jitter, respect provider retry/reset hints where available, and fall back to another LLM provider when the failure is provider-specific.
- 413 / oversized context:** chunk source text before LLM submission and process bounded segments rather than sending a complete document in one request.
- 403 / anti-bot restrictions:** treat blocked sources as failed ingestion attempts, record the failure, and do not fabricate replacement records. Production deployments can add browser automation only where permitted by the source's terms.
- Partial source failure:** one failed source does not terminate the complete pipeline; source-level status is preserved in the coverage report.

7. Storage & Delivery

Artifact	Purpose
export_startups.csv	Startup intelligence
export_products.csv	AI product intelligence
export_research_papers.csv	Research papers + GitHub enrichment
export_jobs.csv	24-hour fresh AI jobs
export_news.csv	24-hour fresh AI news
export_entity_mapping_log.csv	Canonical entity mappings
signal_source_report.json	Live source/freshness/quality diagnostics

8. Evaluation & Reproducibility Checklist

A clean evaluator run should be able to install requirements, execute the pipeline, inspect the six required CSV datasets, verify source URLs and timestamps, inspect the entity mapping log, and read this document for the 500k+ scaling strategy.

Current-run note: jobs/news are time-sensitive and their counts change between runs. A source that has no qualifying item inside the 24-hour window is reported as such; the system does not manufacture records to force 5/5 source coverage.

9. Engineering Principles

Traceability over fabricated completeness. Every retained signal should be attributable to a legitimate source.

Validation before export. Freshness, schema, and duplicate checks happen before final CSV generation.

Graceful degradation. Provider/source failures are isolated and observable instead of silently corrupting the dataset.

Scale by infrastructure. The logical stages and record contracts remain stable while workers, queues, and storage scale horizontally.

Generated by `src/utls/generate_pdf.py`